



HERRAMIENTAS BÁSICAS EN TCP/IPPING, FPING, PINGINFOVIEW Y ARP

PAR P503



21 DE MARZO DE 2026
ANDRÉS VIZCAÍNO BAZAGA
DIRIGIDO AL PROFESOR DON ALFREDO ABAD DOMINGO

ÍNDICE

INTRODUCCIÓN	2
Activación de Sniffers y estado inicial de la caché ARP	5
Comunicación Ping y actualización de la caché ARP	8
Borrado de entradas en la caché ARP.....	10
Comportamiento de ARP ante direcciones inexistentes y entradas estáticas	12
Redirección de tráfico y análisis de capas	14
Análisis de Wireshark	16
CONCLUSIONES	19

INTRODUCCIÓN

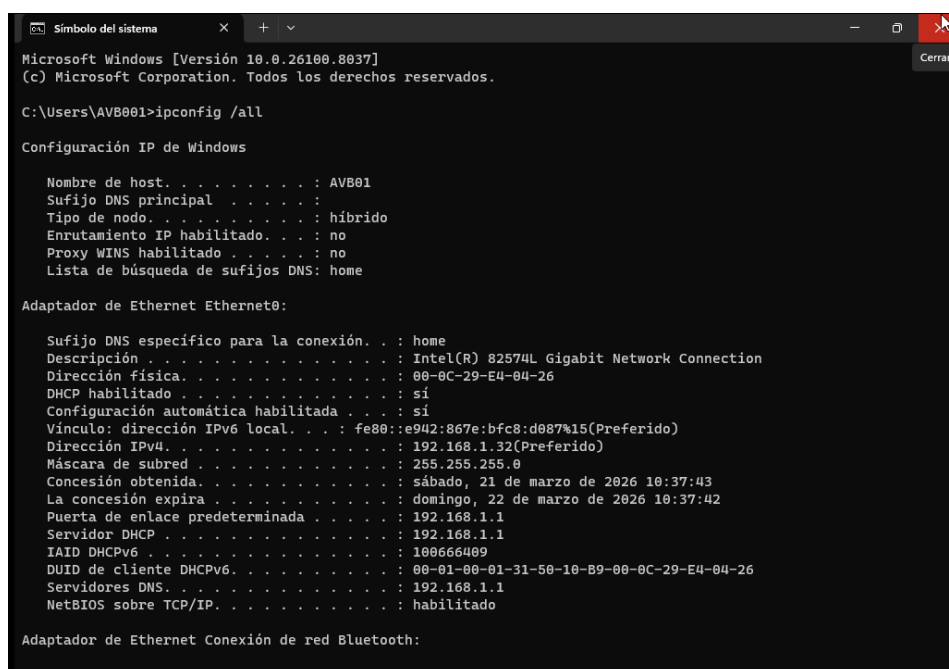
Para el desarrollo de esta práctica contamos con dos nodos configurados en la misma subred. Las máquinas se ejecutan sobre el hipervisor VMware Workstation Pro, habiendo configurado sus adaptadores de red en modo "Bridged" (Puente). Esto permite que el router local de la red física asigne direcciones IP válidas a ambas máquinas virtuales, asegurando que sean accesibles entre sí.

El primer nodo, denominado AVB001 (PC1), ejecuta el sistema operativo Windows 11. Para obtener la dirección MAC (nivel 2) y la dirección IP (nivel 3) de esta máquina, hemos utilizado la consola de comandos (Símbolo del sistema) y ejecutado el comando estándar:

- `ipconfig /all`

Los parámetros obtenidos para el Nodo 1 son:

- **Sistema Operativo:** Windows 11
- **Dirección IPv4:** 192.168.1.32
- **Dirección MAC (Física):** 00-0C-29-E4-04-26



```

Microsoft Windows [Versión 10.0.26100.8037]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\AVB001>ipconfig /all

Configuración IP de Windows

Nombre de host. . . . . : AVB01
Sufijo DNS principal . . . . :
Tipo de nodo. . . . . : híbrido
Enrutamiento IP habilitado. . . : no
Proxy WINS habilitado . . . . : no
Lista de búsqueda de sufijos DNS: home

Adaptador de Ethernet Ethernet0:

Sufijo DNS específico para la conexión. . : home
Descripción . . . . . : Intel(R) 82574L Gigabit Network Connection
Dirección física. . . . . : 00-0C-29-E4-04-26
DHCP habilitado . . . . . : sí
Configuración automática habilitada . . . : sí
Vínculo: dirección IPv6 local. . . : fe80::e942:867e:bfc8:d087%15(Preferido)
Dirección IPv4. . . . . : 192.168.1.32(Preferido)
Máscara de subred . . . . . : 255.255.255.0
Concesión obtenida. . . . . : sábado, 21 de marzo de 2026 10:37:43
La concesión expira . . . . . : domingo, 22 de marzo de 2026 10:37:42
Puerta de enlace predeterminada . . . . : 192.168.1.1
Servidor DHCP . . . . . : 192.168.1.1
IAID DHCPv6 . . . . . : 100666409
DUID de cliente DHCPv6. . . . . : 00-01-00-01-31-50-10-B9-00-0C-29-E4-04-26
Servidores DNS. . . . . : 192.168.1.1
NetBIOS sobre TCP/IP. . . . . : habilitado

Adaptador de Ethernet Conexión de red Bluetooth:
  
```

El segundo nodo de nuestra topología, denominado avb02 (PC2), ejecuta el sistema operativo Ubuntu Desktop 24. Para averiguar la configuración de red de

este equipo, hemos accedido a la terminal de Linux y utilizado el comando estándar:

- `ip a`

el cual nos proporciona información detallada sobre todas las interfaces de red del sistema.

Los parámetros obtenidos para el Nodo 2 son:

- **Sistema Operativo:** Ubuntu Desktop 24
- **Dirección IPv4:** 192.168.1.33
- **Dirección MAC (Física):** 00:0c:29:eb:24:1e

```
avb02@avb02-virtual-machine:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:eb:24:1e brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    inet 192.168.1.33/24 brd 192.168.1.255 scope global dynamic noprefixroute ens33
        valid_lft 84876sec preferred_lft 84876sec
    inet6 fe80::48b9:587a:fe9:4b60/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
avb02@avb02-virtual-machine:~$
```

Una vez preparadas ambas máquinas, debemos instalar en ambas el sniffer Whireshark.

Instalación en Windows 11 (PC1)

1. Abrimos el navegador.
2. Vamos a la página oficial: <https://www.wireshark.org/download.html>
3. Descargamos el instalador de **Windows x64**.
4. Ejecutamos el instalador. Le damos a "Siguiente" a todo, pero **¡muy importante!**: nos aseguramos de que la opción para instalar **Npcap** está marcada (suele venir por defecto). Npcap es el motor que le permite a Windows "escuchar" la red.
5. Terminamos la instalación.

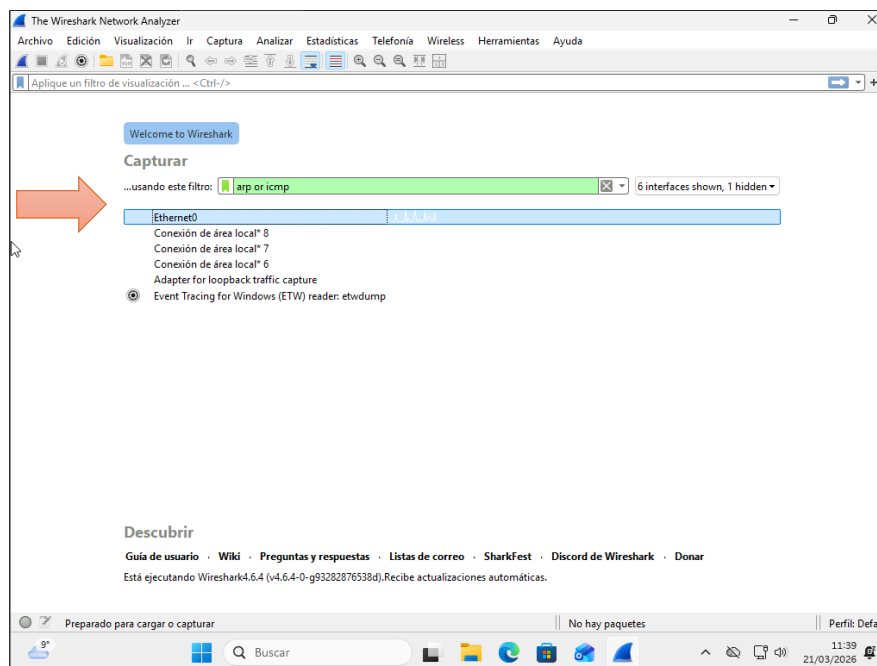
Instalación en Ubuntu Desktop 24 (PC2)

1. Abrimos la Terminal en Ubuntu.
2. `sudo apt install wireshark`
3. Durante la instalación nos saldrá una pantalla azul con letras grises preguntándonos: *"Should non-superusers be able to capture packets?"* (¿Deberían los usuarios que no son root poder capturar paquetes?).
Seleccionamos que SÍ.
4. Para que nuestro usuario tenga permisos reales para capturar sin problemas, escribimos este último comando:
 - `sudo usermod -aG wireshark $USER`
5. **Reiniciamos** para que se apliquen los permisos correctamente.

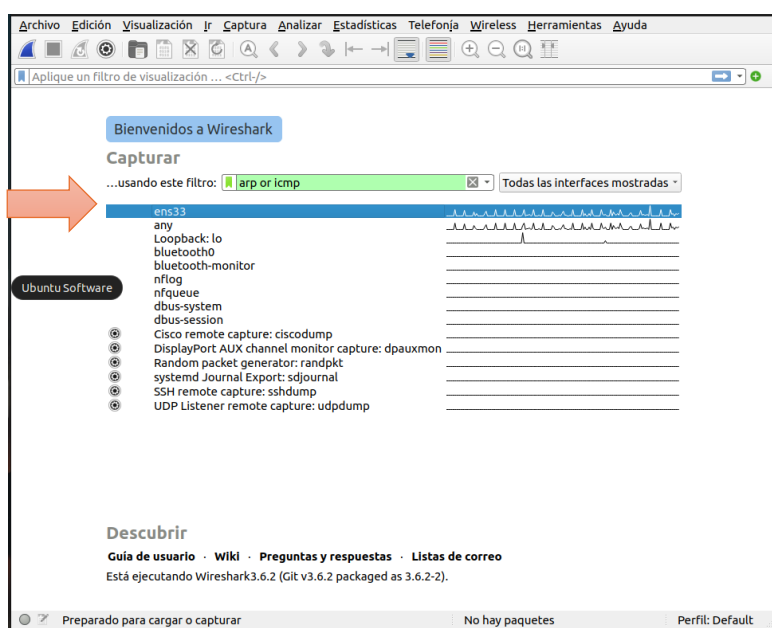
Ahora ya estamos listos para iniciar la práctica.

Activación de Sniffers y estado inicial de la caché ARP

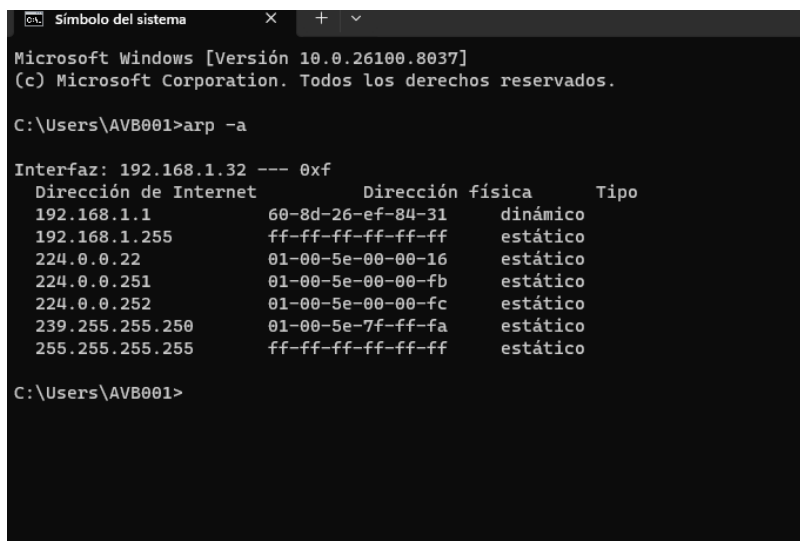
1. **Activamos Wireshark en Windows 11.** Hacemos doble clic en nuestro adaptador de red (Ethernet0) para empezar a capturar. En la barra de filtro de arriba escribe arp or icmp y pulsamos Enter.



2. **Activamos Wireshark en Ubuntu.** Replicamos lo que hemos hecho en Windows.



3. En **Windows 11**, abrimos el Símbolo del sistema (cmd), escribimos
- arp -a
- Revisamos la lista buscando si de casualidad aparece la IP de Ubuntu (192.168.1.33).



```

Microsoft Windows [Versión 10.0.26100.8037]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\AVB001>arp -a

Interfaz: 192.168.1.32 --- 0xf
Dirección de Internet    Dirección física    Tipo
192.168.1.1              60-8d-26-ef-84-31  dinámico
192.168.1.255            ff-ff-ff-ff-ff-ff  estático
224.0.0.22               01-00-5e-00-00-16  estático
224.0.0.251              01-00-5e-00-00-fb  estático
224.0.0.252              01-00-5e-00-00-fc  estático
239.255.255.250          01-00-5e-7f-ff-fa  estático
255.255.255.255          ff-ff-ff-ff-ff-ff  estático

C:\Users\AVB001>

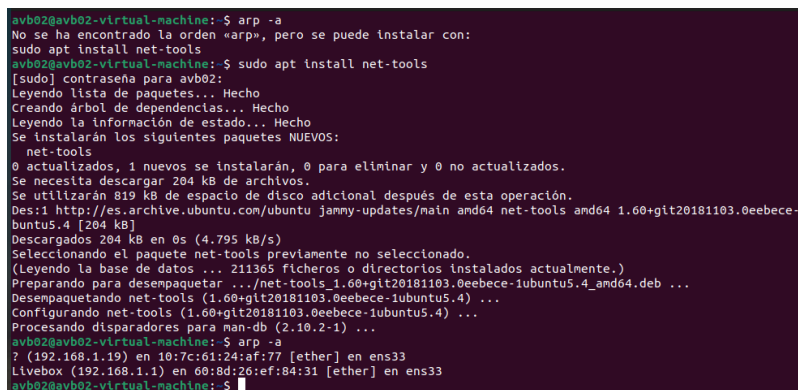
```

4. En **Ubuntu**, abrimos la Terminal, introducimos el comando:

- arp -a.

(Nota: Si Ubuntu te dice que el comando no existe, escribe `sudo apt install net-tools` para instalarlo en un segundo, y luego vuelve a lanzar `arp -a`).

Revisamos si aparece la IP de nuestro Windows (192.168.1.32).



```

avb02@avb02-virtual-machine:~$ arp -a
No se ha encontrado la orden «arp», pero se puede instalar con:
sudo apt install net-tools
avb02@avb02-virtual-machine:~$ sudo apt install net-tools
[sudo] contraseña para avb02:
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes NUEVOS:
net-tools
0 actualizados, 1 nuevos se instalarán, 0 para eliminar y 0 no actualizados.
Se necesita descargar 204 kB de archivos.
Se utilizarán 819 kB de espacio de disco adicional después de esta operación.
Des:1 http://es.archive.ubuntu.com/ubuntu jammy-updates/main amd64 net-tools amd64 1.60+git20181103.0eebece-1ubuntu5.4 [204 kB]
Descargados 204 kB en 0s (4.795 kB/s)
Seleccionando el paquete net-tools previamente no seleccionado.
(Leyendo la base de datos ... 211365 ficheros o directorios instalados actualmente.)
Preparando para desempaquetar .../net-tools_1.60+git20181103.0eebece-1ubuntu5.4_amd64.deb ...
Desempaquetando net-tools (1.60+git20181103.0eebece-1ubuntu5.4) ...
Configurando net-tools (1.60+git20181103.0eebece-1ubuntu5.4) ...
Procesando disparadores para man-db (2.10.2-1) ...
avb02@avb02-virtual-machine:~$ arp -a
? (192.168.1.19) en 10:7c:61:24:af:77 [ether] en ens33
Livebox (192.168.1.1) en 60:8d:26:ef:84:31 [ether] en ens33
avb02@avb02-virtual-machine:~$

```

Como se puede observar en las capturas, ninguno de los dos nodos tiene registrada la dirección IP ni la dirección MAC del otro equipo en su tabla ARP. Esto demuestra que hasta este momento no ha existido comunicación directa entre ellos en la red local.

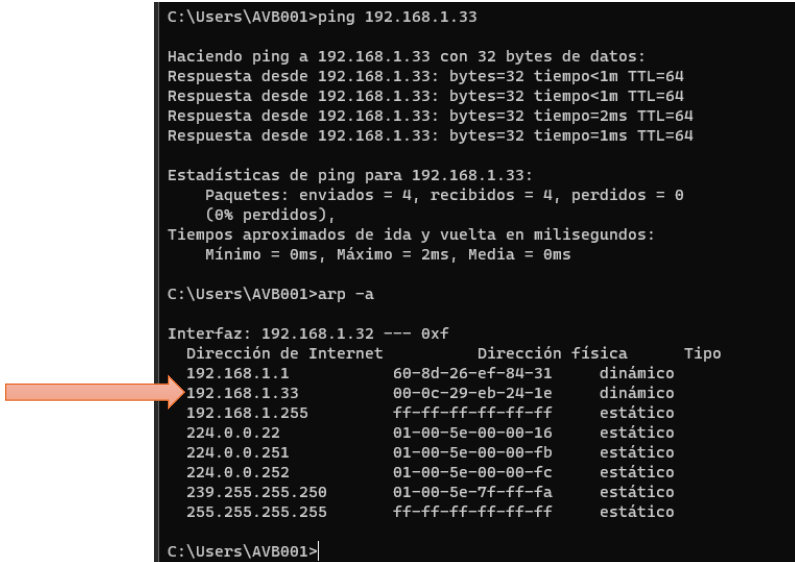
Comunicación Ping y actualización de la caché

ARP

Antes de seguir nos aseguramos de que Wireshark sigue capturando en ambas máquinas.

A continuación, debemos de realizar lo siguiente:

1. En Windows **11**, vamos al Símbolo del sistema y hacemos ping a 192.168.1.33
2. Inmediatamente después, en ese mismo CMD de Windows, volvemos a escribir arp -a y buscamos la IP de Ubuntu en la lista.



```
C:\Users\AVB001>ping 192.168.1.33

Haciendo ping a 192.168.1.33 con 32 bytes de datos:
Respuesta desde 192.168.1.33: bytes=32 tiempo<1m TTL=64
Respuesta desde 192.168.1.33: bytes=32 tiempo<1m TTL=64
Respuesta desde 192.168.1.33: bytes=32 tiempo=2ms TTL=64
Respuesta desde 192.168.1.33: bytes=32 tiempo=1ms TTL=64

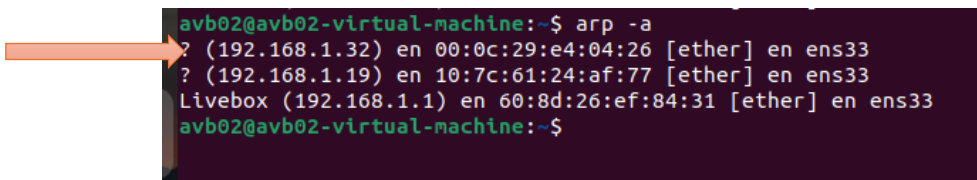
Estadísticas de ping para 192.168.1.33:
    Paquetes: enviados = 4, recibidos = 4, perdidos = 0
              (0% perdidos),
    Tiempos aproximados de ida y vuelta en milisegundos:
        Mínimo = 0ms, Máximo = 2ms, Media = 0ms

C:\Users\AVB001>arp -a

Interfaz: 192.168.1.32 --- 0xf
Dirección de Internet      Dirección física      Tipo
192.168.1.1                60-8d-26-ef-84-31    dinámico
192.168.1.33                00-0c-29-eb-24-1e    dinámico
192.168.1.255              ff-ff-ff-ff-ff-ff    estático
224.0.0.22                 01-00-5e-00-00-16    estático
224.0.0.251                01-00-5e-00-00-fb    estático
224.0.0.252                01-00-5e-00-00-fc    estático
239.255.255.250            01-00-5e-7f-ff-fa    estático
255.255.255.255            ff-ff-ff-ff-ff-ff    estático

C:\Users\AVB001>
```

3. Ahora, vamos a **Ubuntu**, abrimos la terminal y volvemos a escribir arp -a y buscamos la IP de Windows en la lista.



```
avb02@avb02-virtual-machine:~$ arp -a
? (192.168.1.32) en 00:0c:29:e4:04:26 [ether] en ens33
? (192.168.1.19) en 10:7c:61:24:af:77 [ether] en ens33
Livebox (192.168.1.1) en 60:8d:26:ef:84:31 [ether] en ens33
avb02@avb02-virtual-machine:~$
```

4. Una vez realizados todos estos pasos, vamos a Wireshark en ambas máquinas y pulsamos el **cuadrado rojo** de la esquina superior izquierda para detener las capturas.

¿En qué han variado las cachés?

Al ejecutar nuevamente `arp -a`, observamos que la caché de ambos nodos ha variado añadiendo una nueva entrada. En Windows 11 ha aparecido la IP y la MAC de Ubuntu, y de forma recíproca, en Ubuntu ha aparecido la IP y la MAC de Windows 11.

¿Por qué ha ocurrido esto?

Esto sucede por el propio funcionamiento de la pila TCP/IP. Para que Windows pudiera enviar el "Ping" (Nivel 3) a la IP de Ubuntu, necesitaba encapsularlo en una trama Ethernet (Nivel 2), para lo cual necesitaba obligatoriamente conocer la dirección MAC de destino. Al no tenerla en su caché inicial, Windows detuvo momentáneamente el Ping y envió un mensaje *ARP Request* (difusión/broadcast) a toda la red preguntando: *"¿Quién tiene la IP 192.168.1.33?"*. Ubuntu lo escuchó y respondió directamente a Windows con un *ARP Reply* diciendo: *"Soy yo, y esta es mi MAC"*. En ese intercambio, ambos equipos aprovecharon para registrar y guardar en su memoria caché la información del otro (IP asociada a su MAC) para agilizar futuras comunicaciones.

Borrado de entradas en la caché ARP

El objetivo de este paso es ver qué pasa cuando borramos la información que un equipo ha aprendido y qué impacto tiene esto en el otro equipo.

Para ello realizaremos los siguientes pasos:

1. En **Windows 11**, abrimos el Símbolo del sistema **como Administrador** (muy importante para poder borrar).
2. Escribimos el siguiente comando para borrar la entrada de Ubuntu:
 - `arp -d 192.168.1.33`.
3. Escribimos `arp -a` en Windows para comprobar que efectivamente la IP 192.168.1.33 ha desaparecido de la lista.

```

Administrador: Símbolo del sistema
Microsoft Windows [Versión 10.0.26100.8037]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Windows\System32>arp -d 192.168.1.33

C:\Windows\System32>arp -a

Interfaz: 192.168.1.32 --- 0xf
Dirección de Internet    Dirección física    Tipo
192.168.1.1              60-8d-26-ef-84-31  dinámico
192.168.1.255            ff-ff-ff-ff-ff-ff  estático
224.0.0.22               01-00-5e-00-00-16  estático
224.0.0.251              01-00-5e-00-00-fb  estático
224.0.0.252              01-00-5e-00-00-fc  estático
239.255.255.250          01-00-5e-7f-ff-fa  estático
255.255.255.255          ff-ff-ff-ff-ff-ff  estático

C:\Windows\System32>
  
```

4. Ahora, vamos a **Ubuntu** y ejecutamos `arp -a`.

```

avb02@avb02-virtual-machine:~$ arp -a
? (192.168.1.32) en 00:0c:29:e4:04:26 [ether] en ens33
? (192.168.1.19) en 10:7c:61:24:af:77 [ether] en ens33
Livebox (192.168.1.1) en 60:8d:26:ef:84:31 [ether] en ens33
avb02@avb02-virtual-machine:~$
  
```

¿Ha variado la caché de Windows? Y, ¿la de Ubuntu? ¿Por qué?

Al consultar nuevamente las tablas, comprobamos que la caché de PC1 (Windows) efectivamente ha variado: la entrada correspondiente a la IP 192.168.1.33 ha sido eliminada. Sin embargo, la caché de PC2 (Ubuntu) permanece inalterada y sigue conservando la entrada de Windows (192.168.1.32). Esto se debe a que las cachés ARP son memorias **locales** e independientes en cada equipo. Borrar un registro en un nodo no envía ninguna notificación ni afecta a las memorias de los demás dispositivos de la red.

¿Qué tenemos que hacer para volver a recuperar el registro de pc2 en la caché ARP de pc1?

Para que PC1 vuelva a registrar la dirección MAC de PC2, es necesario forzar una nueva comunicación que requiera resolución de Nivel 2. La forma más sencilla es generar tráfico dirigido hacia PC2, por ejemplo, volviendo a ejecutar un ping 192.168.1.33 desde PC1. Al no tener la MAC en caché, PC1 emitirá automáticamente un nuevo *ARP Request*, recibirá el *ARP Reply* de PC2 y volverá a almacenar la entrada en su memoria local.

Comportamiento de ARP ante direcciones inexistentes y entradas estáticas

Acciones a realizar en Windows:

1. En CMD de Windows, hacemos un ping a la IP 10.10.1.1 y esperamos a que termine.
2. Escribimos arp -a para comprobar si esa IP se ha guardado.

```
C:\Windows\System32>ping 10.10.1.1

Haciendo ping a 10.10.1.1 con 32 bytes de datos:
Tiempo de espera agotado para esta solicitud.
Tiempo de espera agotado para esta solicitud.
Tiempo de espera agotado para esta solicitud.
Tiempo de espera agotado para esta solicitud.

Estadísticas de ping para 10.10.1.1:
    Paquetes: enviados = 4, recibidos = 0, perdidos = 4
              (100% perdidos),

C:\Windows\System32>arp -a

Interfaz: 192.168.1.32 --- 0xf
Dirección de Internet      Dirección física      Tipo
192.168.1.1                60-8d-26-ef-84-31    dinámico
192.168.1.255              ff-ff-ff-ff-ff-ff    estático
224.0.0.22                 01-00-5e-00-00-16    estático
224.0.0.251                01-00-5e-00-00-fb    estático
224.0.0.252                01-00-5e-00-00-fc    estático
239.255.255.250            01-00-5e-7f-ff-fa    estático
255.255.255.255            ff-ff-ff-ff-ff-ff    estático

C:\Windows\System32>
```

3. Ahora vamos a inventarnos una entrada estática.
En el mismo CMD que ya tenemos abierto, escribimos el siguiente comando:
 - arp -s 10.10.1.1 00-00-00-01-02-03
4. Volvemos a escribir arp -a y buscamos en la lista.
5. Volvemos a hacer ping a 10.10.1.1

```
C:\Windows\System32>arp -s 10.10.1.1 00-00-00-01-02-03

C:\Windows\System32>arp -a

Interfaz: 192.168.1.32 --- 0xf
Dirección de Internet      Dirección física      Tipo
10.10.1.1                  00-00-00-01-02-03    estático
192.168.1.1                60-8d-26-ef-84-31    dinámico
192.168.1.255              ff-ff-ff-ff-ff-ff    estático
224.0.0.22                 01-00-5e-00-00-16    estático
224.0.0.251                01-00-5e-00-00-fb    estático
224.0.0.252                01-00-5e-00-00-fc    estático
239.255.255.250            01-00-5e-7f-ff-fa    estático
255.255.255.255            ff-ff-ff-ff-ff-ff    estático

C:\Windows\System32>ping 10.10.1.1

Haciendo ping a 10.10.1.1 con 32 bytes de datos:
Tiempo de espera agotado para esta solicitud.
Tiempo de espera agotado para esta solicitud.
Tiempo de espera agotado para esta solicitud.
Tiempo de espera agotado para esta solicitud.

Estadísticas de ping para 10.10.1.1:
    Paquetes: enviados = 4, recibidos = 0, perdidos = 4
              (100% perdidos),

C:\Windows\System32>
```

A continuación, vamos a analizar los resultados obtenidos en los pasos 1 y 2:

- **Resultado del primer ping:** El comando no obtiene respuesta.
- **Consulta ARP:** Al consultar la caché con `arp -a`, comprobamos que no existe ninguna entrada para la IP 10.10.1.1. Al no existir el equipo, nadie respondió al *ARP Request*, por lo que Windows no pudo registrar nada.

A continuación, vamos a analizar los resultados obtenidos en los pasos 3,4 y 5:

- **¿El mensaje proporcionado por el nuevo ping es el mismo que el anterior?**

Sí, el mensaje de error de ping es el mismo

- **Al consultar de nuevo la caché ARP buscando la entrada 10.10.1.1**

Ahora sí aparece registrada en la tabla, marcada con el tipo "estático", ya que la hemos introducido manualmente.

- **¿Por qué, a pesar de que ARP tiene una entrada para el destino, ping sigue sin funcionar?**

El ping sigue sin funcionar porque la tabla ARP solo resuelve el problema de la Capa 2 (saber a qué dirección MAC enviar la trama). Windows confía en nuestra entrada estática, empaqueta el ICMP Echo Request y lo lanza a la red con destino a la MAC 00:00:00:01:02:03. Sin embargo, como esa tarjeta de red física no existe realmente en nuestra red, nadie recoge ese paquete y, por tanto, nadie envía el mensaje de vuelta (ICMP Echo Reply). El ping agota su tiempo de espera esperando una respuesta que nunca llegará.

Redirección de tráfico y análisis de capas

Lo que debemos de hacer es:

1. En CMD (como Administrador), borramos la entrada falsa que creamos antes:
 - `arp -d 10.10.1.1`
2. Por si acaso ha reaparecido, borramos también la de Ubuntu:
 - `arp -d 192.168.1.33`
3. Escribimos `arp -a` para confirmar que está todo limpio.

```
avb02@avb02-virtual-machine:~$ arp -d 192.168.1.33
SIOCDARP(dontpub): Operación no permitida
avb02@avb02-virtual-machine:~$ arp -a
? (192.168.1.32) en 00:0c:29:e4:04:26 [ether] en ens33
? (192.168.1.19) en 10:7c:61:24:af:77 [ether] en ens33
Livebox (192.168.1.1) en 60:8d:26:ef:84:31 [ether] en ens33
avb02@avb02-virtual-machine:~$
```

4. Ahora, añadimos la nueva entrada asociando la IP falsa a la MAC *real* de Ubuntu:
 - `arp -s 10.10.1.1 00-0c-29-eb-24-1e`
5. Volvemos a hacer ping 10.10.1.1

```
C:\Windows\System32>arp -d 10.10.1.1

C:\Windows\System32>arp -s 10.10.1.1 00-0c-29-eb-24-1e

C:\Windows\System32>ping 10.10.1.1

Haciendo ping a 10.10.1.1 con 32 bytes de datos:
Tiempo de espera agotado para esta solicitud.
Tiempo de espera agotado para esta solicitud.
Tiempo de espera agotado para esta solicitud.
Tiempo de espera agotado para esta solicitud.

Estadísticas de ping para 10.10.1.1:
    Paquetes: enviados = 4, recibidos = 0, perdidos = 4
              (100% perdidos),
C:\Windows\System32>
```

Vamos ahora a analizar nuestras pruebas y nuestros resultados obtenidos:

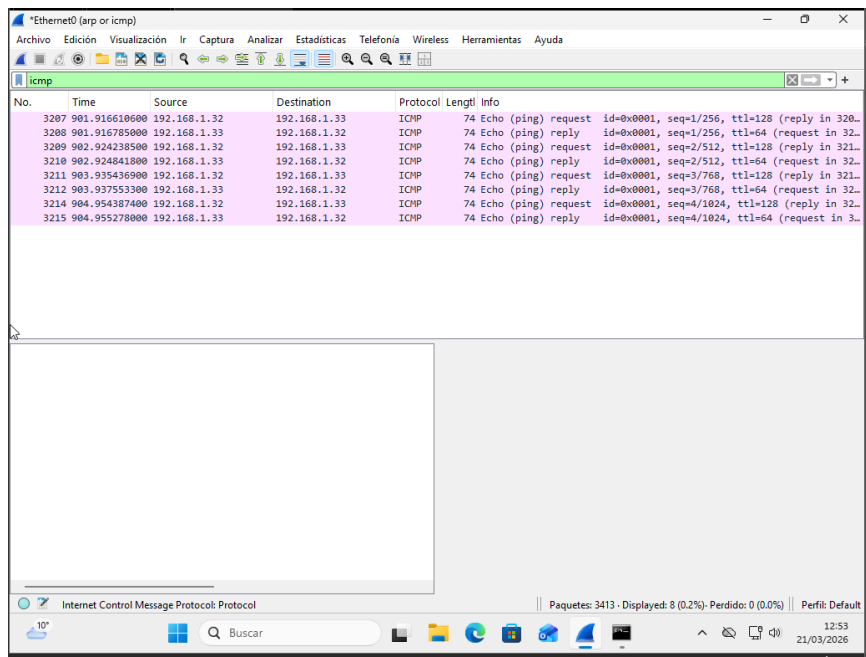
Al hacer ping a 10.10.1.1 desde PC1, ¿Qué ocurre? ¿Por qué?

Lo que ocurre es que el ping **vuelve a fallar** (Tiempo de espera agotado), a pesar de que el paquete esta vez sí llega físicamente a la tarjeta de red de PC2.

Esto se justifica analizando el comportamiento de las capas del modelo OSI/TCP-IP:

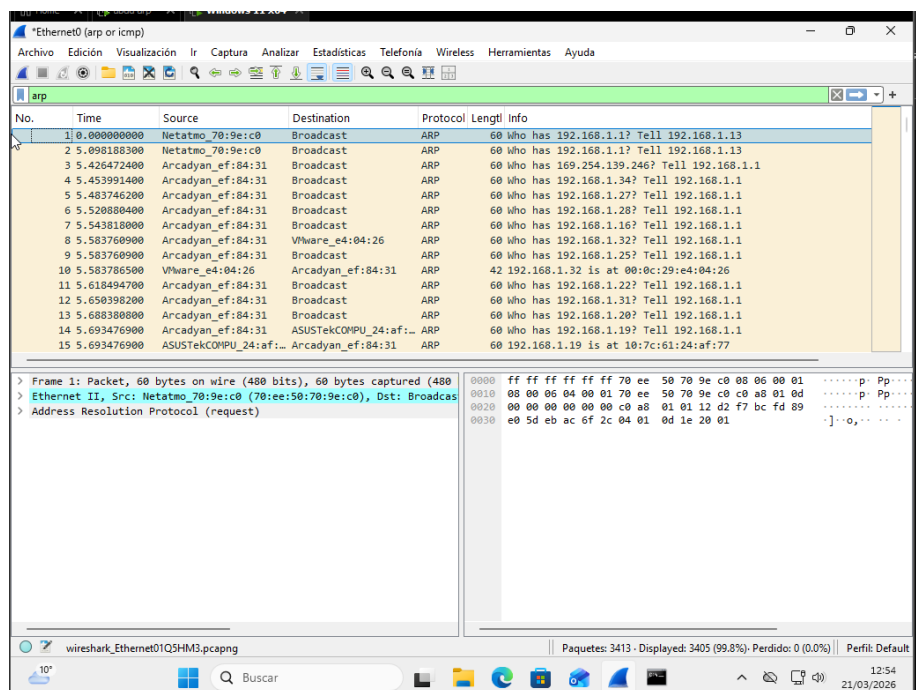
1. **Capa 2 (Enlace):** La operación es un éxito. Windows consulta su tabla ARP, ve que la IP 10.10.1.1 está en la MAC de Ubuntu y envía la trama Ethernet. La tarjeta de red de Ubuntu ve su propia MAC en el destino, acepta la trama y se la pasa al sistema operativo.
2. **Capa 3 (Red):** La operación fracasa. El protocolo IP de Ubuntu recibe el paquete, lee la cabecera y ve que la IP de destino es 10.10.1.1. Como la IP configurada en Ubuntu es 192.168.1.33, el sistema operativo determina que ese paquete no es para él (se lo han entregado por error) y lo descarta silenciosamente. Al ser descartado, Ubuntu nunca genera el mensaje de respuesta "ICMP Echo Reply", y por tanto, el ping en Windows acaba dando error por agotamiento de tiempo.

Análisis de Wireshark



Wireshark capture of ICMP packets. The packet list shows several Echo (ping) requests and replies between 192.168.1.32 and 192.168.1.33. The packet details pane shows the selected packet (No. 3207) as an Echo (ping) request with ID 0x0001, sequence 1/256, and TTL 128.

No.	Time	Source	Destination	Protocol	Length	Info
3207	991.916610600	192.168.1.32	192.168.1.33	ICMP	74	Echo (ping) request id=0x0001, seq=1/256, ttl=128 (reply in 320...
3208	991.916785000	192.168.1.33	192.168.1.32	ICMP	74	Echo (ping) reply id=0x0001, seq=1/256, ttl=64 (request in 32...
3209	992.924238500	192.168.1.32	192.168.1.33	ICMP	74	Echo (ping) request id=0x0001, seq=2/512, ttl=128 (reply in 321...
3210	992.924841800	192.168.1.33	192.168.1.32	ICMP	74	Echo (ping) reply id=0x0001, seq=2/512, ttl=64 (request in 32...
3211	993.935436900	192.168.1.32	192.168.1.33	ICMP	74	Echo (ping) request id=0x0001, seq=3/768, ttl=128 (reply in 321...
3212	993.937553300	192.168.1.33	192.168.1.32	ICMP	74	Echo (ping) reply id=0x0001, seq=3/768, ttl=64 (request in 32...
3214	994.954387400	192.168.1.32	192.168.1.33	ICMP	74	Echo (ping) request id=0x0001, seq=4/1024, ttl=128 (reply in 32...
3215	994.955278000	192.168.1.33	192.168.1.32	ICMP	74	Echo (ping) reply id=0x0001, seq=4/1024, ttl=64 (request in 3...



Wireshark capture of ARP packets. The packet list shows several ARP requests and replies. The packet details pane shows the selected packet (No. 1) as an ARP request for 192.168.1.1, with source MAC 08:00:00:00:00:00 and destination MAC ff:ff:ff:ff:ff:ff.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	Netatmo_70:9e:c0	Broadcast	ARP	60	Who has 192.168.1.1? Tell 192.168.1.13
2	5.098188300	Netatmo_70:9e:c0	Broadcast	ARP	60	Who has 192.168.1.1? Tell 192.168.1.13
3	5.426472400	Arcadyan_ef:84:31	Broadcast	ARP	60	Who has 169.254.139.246? Tell 192.168.1.1
4	5.453991400	Arcadyan_ef:84:31	Broadcast	ARP	60	Who has 192.168.1.34? Tell 192.168.1.1
5	5.483746200	Arcadyan_ef:84:31	Broadcast	ARP	60	Who has 192.168.1.27? Tell 192.168.1.1
6	5.520880400	Arcadyan_ef:84:31	Broadcast	ARP	60	Who has 192.168.1.28? Tell 192.168.1.1
7	5.543818000	Arcadyan_ef:84:31	Broadcast	ARP	60	Who has 192.168.1.16? Tell 192.168.1.1
8	5.583760900	Arcadyan_ef:84:31	Broadcast	ARP	60	Who has 192.168.1.32? Tell 192.168.1.1
9	5.583760900	Arcadyan_ef:84:31	Broadcast	ARP	60	Who has 192.168.1.25? Tell 192.168.1.1
10	5.583786500	VMware_e4:04:26	Arcadyan_ef:84:31	ARP	42	192.168.1.32 is at 00:0c:29:e4:04:26
11	5.618494700	Arcadyan_ef:84:31	Broadcast	ARP	60	Who has 192.168.1.22? Tell 192.168.1.1
12	5.650398200	Arcadyan_ef:84:31	Broadcast	ARP	60	Who has 192.168.1.31? Tell 192.168.1.1
13	5.688380800	Arcadyan_ef:84:31	Broadcast	ARP	60	Who has 192.168.1.20? Tell 192.168.1.1
14	5.693476900	ASUSTekCOMPU_24:af:...	Arcadyan_ef:84:31	ARP	60	Who has 192.168.1.19? Tell 192.168.1.1
15	5.693476900	ASUSTekCOMPU_24:af:...	Arcadyan_ef:84:31	ARP	60	192.168.1.19 is at 10:7c:61:24:af:77

Frame 1: Packet, 60 bytes on wire (480 bits), 60 bytes captured (480) on interface 0 (eth0) by Wireshark 2.10.2 (Ubuntu 22.04 LTS)

Ethernet II, Src: Netatmo_70:9e:c0 (70:ee:50:70:9e:c0), Dst: Broadcast

Address Resolution Protocol (request)

0000 ff ff ff ff ff ff 70 ee 50 70 9e c0 08 06 00 01p..Pp...

0010 08 00 06 04 00 01 70 ee 50 70 9e c0 c0 a8 01 0dp..Pp...

0020 00 00 00 00 00 c0 a8 01 01 12 d2 f7 bc fd 89Pp..Pp...

0030 e0 5d eb ac 6f 2c 04 01 0d 1e 20 01]..o,.....

Al revisar las capturas de red realizadas con Wireshark durante las operaciones, hemos podido localizar claramente los paquetes correspondientes a los protocolos ARP y PING (ICMP).

¿Cuántos datagramas intervienen en la resolución ARP?

En una resolución ARP estándar intervienen **dos** datagramas o tramas principales:

1. **ARP Request (Petición):** El equipo origen pregunta a toda la red quién tiene una IP específica.
2. **ARP Reply (Respuesta):** El equipo destino responde directamente al origen entregándole su dirección MAC.

Secuencia de tramas involucradas en la resolución:

Tomando como ejemplo la comunicación entre nuestro PC1 (Windows: 192.168.1.32) y PC2 (Ubuntu: 192.168.1.33), la secuencia teórica que se desencadena al no conocer la MAC es la siguiente:

- **Trama 1 (ARP Request):**
 - *MAC Origen:* 00:0c:29:e4:04:26 (Windows)
 - *MAC Destino:* FF:FF:FF:FF:FF:FF (Dirección de Broadcast, para que el switch lo replique a todos los equipos de la red local).
 - *Mensaje:* "Who has 192.168.1.33? Tell 192.168.1.32" (¿Quién tiene la IP 192.168.1.33? Decídselo a la IP 192.168.1.32).
- **Trama 2 (ARP Reply):**
 - *MAC Origen:* 00:0c:29:eb:24:1e (Ubuntu)
 - *MAC Destino:* 00:0c:29:e4:04:26 (Windows - Es una respuesta Unicast, directa de vuelta al origen).
 - *Mensaje:* "192.168.1.33 is at 00:0c:29:eb:24:1e" (Yo tengo la IP 192.168.1.33 y mi MAC es 00:0c:29:eb:24:1e).

¿Qué tipo de paquetes identifican los envíos ping? ¿Y las resoluciones ARP?

- Como se observa en la primera captura, los envíos *ping* se identifican con paquetes del protocolo **ICMP** (Internet Control Message Protocol). Concretamente, se utilizan los mensajes *Echo (ping) request* (para la petición) y *Echo (ping) reply* (para la respuesta).
- Las resoluciones de direcciones físicas identifican sus paquetes con el propio protocolo **ARP** (Address Resolution Protocol), operando directamente sobre la capa de enlace (Ethernet).

¿Por qué a pesar de hacer lo mismo no se ejecuta la resolución ARP al hacer un segundo ping inmediato?

Porque el protocolo ARP utiliza una memoria caché local para ser eficiente. Al hacer el primer ping, la resolución IP-MAC (192.168.1.33 --> 00:0c:29:eb:24:1e) se almacena en la tabla ARP del sistema operativo durante un tiempo determinado. Al lanzar un segundo ping inmediatamente después, Windows consulta primero esta caché, encuentra la MAC correspondiente y envía el paquete ICMP directamente, ahorrándose tener que inundar la red local con un nuevo mensaje de Broadcast (ARP Request).

CONCLUSIONES

Durante el desarrollo de la práctica hemos comprobado las siguientes diferencias de comportamiento entre ambos sistemas operativos:

- **Comando ping:** En Windows, el comando envía por defecto 4 paquetes de petición (Echo Request) y finaliza mostrando unas estadísticas. En Linux (Ubuntu), el comando se ejecuta en un bucle infinito enviando paquetes hasta que el usuario lo detiene manualmente.
- **Comando arp:**
 - **Visualización:** Aunque arp -a funciona en ambos, Windows muestra una tabla con columnas muy definidas (Dirección de Internet, Dirección física, Tipo), mientras que Linux lo muestra en formato de texto continuo indicando la interfaz.
 - **Privilegios:** Para modificar la tabla ARP (añadir con -s o borrar con -d), Windows requiere abrir el Símbolo del sistema obligatoriamente con privilegios de Administrador. En Linux, realizar estas mismas acciones de escritura/borrado exige el uso del prefijo sudo para escalar privilegios temporales de *root*.